

Bản đồ cây

TÓM TẮT

Đây là bản đồ điều hướng của cả cây: nó nói cây gồm gì, vì sao được chia thành sáu phần theo thứ tự đó, và bốn nguyên lý sẽ gặp lại ở gần như mọi chương. Đọc xong bạn biết nên bắt đầu từ đâu, phần nào đáng đầu tư sâu, phần nào chỉ cần tra khi cần, và phần nào sẽ hết hạn trước tiên. Nếu chỉ nhớ một điều: cây được tổ chức theo *mạch vấn đề dẫn tới giải pháp* chứ không theo danh mục thuật ngữ, vì đó là khác biệt giữa cách người mới và cách chuyên gia phân loại bài toán.

Cây này được dựng từ một lộ trình tự soạn, với một mục tiêu hẹp và cụ thể: *đọc hiểu paper, hiểu vì sao tác giả làm như vậy, và gọi tên được trade-off của mỗi kỹ thuật*. Nó không nhằm biến người đọc thành người thuộc lòng danh mục thuật ngữ. Sự khác biệt giữa hai mục tiêu ấy quyết định toàn bộ cách trình bày phía sau, nên đáng nói rõ ngay từ đầu.

Vì sao trình bày theo mạch chứ không theo danh mục (Why a Problem-Driven Narrative)

Có một quan sát lặp lại trong nghiên cứu về chuyên gia: người mới phân loại bài toán theo *đặc điểm bề mặt* — tên module, tên layer, con số trong bảng — còn chuyên gia phân loại theo *nguyên lý sâu* — vấn đề nào đang được giải, giả định nào đang được đặt. Một danh sách gạch đầu dòng dạy rất tốt cách phân loại thứ nhất và rất tệ cách thứ hai. Do đó phần lớn cây này được viết theo mạch **vấn đề** → **giải pháp** → **giả định** → **giá phải trả** → **vấn đề mới sinh ra**, và các kỹ thuật cùng họ được đi qua *theo đúng thứ tự lịch sử chúng ra đời để gỡ nút thắt của nhau*.

Cách đọc kèm theo cũng khác. Mạch suy luận đã được viết sẵn ở đây chính là một *worked example* theo nghĩa của tâm lý học nhận thức: việc của người đọc không phải là tự khám phá lại, mà là *dự đoán bước tiếp theo trước khi đọc nó*. Nếu dự đoán đúng thì bạn đã nắm nguyên lý; nếu sai thì chỗ sai chính là chỗ đáng dừng lại.

Phân tầng theo chu kỳ bán rã của tri thức (Knowledge Half-Life Tiers)

Không phải thứ gì trong lĩnh vực này cũng đáng học sâu như nhau, vì chúng hết hạn với tốc độ rất khác nhau. Cây dùng bốn nhãn, và nhãn xuất hiện ngay ở tiêu đề mỗi chương.

Bảng 1. Bốn tầng bán rã, mức đầu tư tương ứng, và vị trí trong cây.

Tầng	Nội dung	Mức đầu tư	Nằm ở đâu
[T1]	Toán, vật lý, first principle	Chỉ học phần cần thiết, tới mức dùng được và giải thích được; không chứng minh định lý	Phần A
[T2]	Nguyên lý thiết kế hệ thống, giới hạn tính toán, trade-off	Trọng tâm chính — đây là thứ còn giá trị sau mười năm	Phần B, E
[T3]	Thuật toán và mô hình	Trọng tâm chính — học ý tưởng cốt lõi và lý do ra đời, không học thuộc danh sách	Phần C, F
[T4]	Thư viện, framework, công cụ	Học đủ để làm việc, không đầu tư sâu; tra khi cần	Phần D

Bảng 1 giải thích một điều thoát nhìn có vẻ mâu thuẫn: các bài tập tự viết backward pass và tự chạy gradient check, dù trông giống bài tập công cụ, thực ra thuộc tầng [T2]–[T3]. Chúng xây trực giác về đồ thị tính toán, về nơi gradient chết, về cái giá bộ nhớ của activation — những thứ không hết hạn. Ngược lại, một danh sách kiến trúc cụ thể sẽ cũ trong vài năm và nên được đọc như lịch sử tư duy, không như trạng thái hiện tại.

Vì sao chia thành sáu phần, và thứ tự phụ thuộc (The Six Parts)

Sáu phần không phải sáu chủ đề song song mà là sáu tầng câu hỏi nối tiếp nhau. Phần A trả lời *ảnh từ đâu ra và ngôn ngữ toán nào dùng để nói về nó*; nó ngắn có chủ ý vì nguyên tắc là học theo nhu cầu. Phần B trả lời một câu hỏi duy nhất và là câu hỏi bị bỏ qua nhiều nhất: *làm sao ra quyết định thiết kế đúng khi chưa có mô hình nào chạy?* Mọi chương trong đó là một dạng ràng buộc, và mọi kỹ thuật xuất hiện đều là câu trả lời cho một thất bại cụ thể trước đó. Chỉ sau khi có Phần B thì Phần C — các họ thuật toán và mô hình — mới đọc được như một chuỗi lựa chọn có lý do, thay vì một danh mục kiến trúc. Phần D là tầng công cụ và triển khai, cố ý đặt sau vì nó hết hạn nhanh nhất. Phần E đóng vòng phương pháp: đọc paper, thiết kế thí nghiệm, đánh giá độ tin cậy và làm dự án xuyên suốt. Phần F là phần áp dụng, và nó chỉ đọc được sau năm phần kia: nó hỏi một câu duy nhất, rằng có nên thay một khối trong hệ SLAM cổ điển bằng một mạng hay không, và nếu có thì khối nào.

Bảng 2. Sáu phần, câu hỏi mỗi phần trả lời, và mức độ ưu tiên.

	Phần	Câu hỏi nó trả lời	Mức
A	Nền tảng có giới hạn	Ảnh từ đâu ra, và cần đúng bao nhiêu toán để nói về nó?	nền, học vừa đủ
B	Tư duy hệ thống	Ra quyết định thiết kế đúng thể nào khi chưa có mô hình nào chạy?	cốt lõi
C	Thuật toán và mô hình	Các họ mô hình ra đời để gỡ nút thắt nào của nhau?	cốt lõi
D	Kỹ thuật, tối ưu, triển khai	Từ mô hình chạy được tới hệ thống chạy thật thì mất gì?	công cụ
E	Đánh giá, độ tin cậy, dự án	Làm sao biết mình không tự lừa mình?	cốt lõi
F	Học sâu cho visual SLAM	Có nên thay một khối cổ điển bằng mạng, và nếu có thì khối nào?	áp dụng

Bảng 2 cũng là thứ tự đọc đề xuất cho lần đầu, với một ngoại lệ: Phần F chỉ đáng đọc khi bạn đã có một hệ SLAM thật đang đau ở một chỗ cụ thể, và khi đó nên đọc chương 27 trước để lọc bẫy

chương còn lại. Từ lần thứ hai trở đi, cách đọc hiệu quả hơn là trộn các nhánh xa nhau và tự kiểm tra chéo, vì gần như mọi chương trong cây đều là một cách nhìn khác của cùng vài nguyên lý.

Vài nguyên lý xuyên suốt, gặp lại ở nhiều chương (Recurring Principles)

Bốn ý dưới đây xuất hiện lại ở những chỗ tưởng như không liên quan, và nhận ra chúng là dấu hiệu đã đọc đúng. Thứ nhất, **độ khó không biến mất, nó chỉ bị đẩy đi chỗ khác** — sang dữ liệu, sang hậu xử lý, sang thời gian huấn luyện, hay sang một giả định về đầu vào; câu hỏi luôn đáng hỏi là *họ đã đẩy nó đi đâu*. Thứ hai, trục đánh đổi trung tâm **giả định mạnh cộng ít dữ liệu** đối lại **giả định yếu cộng nhiều dữ liệu**, giải thích gần như mọi bước tiến mười lăm năm qua và sẽ gặp lại ở CNN đối lại ViT, ở hình học cổ điển đối lại mô hình feed-forward. Thứ ba, **không thể không có giả định**; chỉ có việc chọn giả định nào, và chọn sai thì thêm dữ liệu không cứu được. Thứ tư, **hiểu một hệ thống nghĩa là biết nó hỏng ở đâu**, không phải biết nó chạy được gì — đây là lý do mỗi tài liệu trong cây đều có một mục bắt buộc về giới hạn và trường hợp hỏng.

Cách dùng cây (How to Use This Tree)

Mỗi thư mục có một tệp `00-map.tex` nói nhánh đó gồm gì và vì sao chia như vậy; đó là chỗ nên đọc trước khi vào nội dung. Mỗi tài liệu mở đầu bằng một dòng *Phụ thuộc* cho biết cần đọc gì trước, và kết bằng một mục *Kết nối* trỏ sang các nhánh liên quan cùng một câu giải thích quan hệ. Cuối mỗi tài liệu là ba tới bảy câu hỏi tự kiểm; theo lịch ôn của kho, hãy trả lời chúng *trước* khi mở lại bài, sau một ngày, một tuần, một tháng, ba tháng, và sửa ngay chỗ nào đọc lại thấy không hiểu.

Mỗi chương kết thúc bằng khối *Thực hành* gồm bốn mục: repo và tài nguyên, câu hỏi kiểm tra hiểu, một mini project làm trong vài giờ tới hai ngày, và với những chương đáng đầu tư thì thêm một project một tới ba tuần. Lời khuyên đi kèm là làm mini project của mọi chương, còn project thì chọn lọc — vì đọc mà không tái tạo lại thì chỉ là tiêu thụ.

CẠM BÃY

Phần *Repo* và *SOTA* trong các khối *Thực hành* chốt tại **tháng 9/2026** và là phần hết hạn nhanh nhất của cây: tên mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng. Repo hạ tầng (COLMAP, timm, diffusers, ONNX Runtime, pycocotools) bền hơn nhiều so với tên mô hình. Đọc phần đó như *chỗ để bắt đầu tìm*, không phải *câu trả lời đúng*.

Câu hỏi còn mở (Open Questions)

Ba điều cần xử lý khi quay lại cây này. Thứ nhất, `refs.bib` được soạn từ trí nhớ về các công trình gốc; tên tác giả, năm và hội nghị đủ chính xác để tra cứu, nhưng phải đối chiếu bản gốc trước khi trích dẫn ra ngoài cây. Thứ hai, mọi con số định lượng trong cây (mAP, tốc độ, số tham số, độ lớn hiệu ứng) cần được kiểm lại tại nguồn trước khi dùng để ra quyết định thật. Thứ ba, phần *Repo* và *SOTA* cần được rà lại mỗi sáu tháng; lần rà gần nhất là lần soạn, tháng 9/2026.